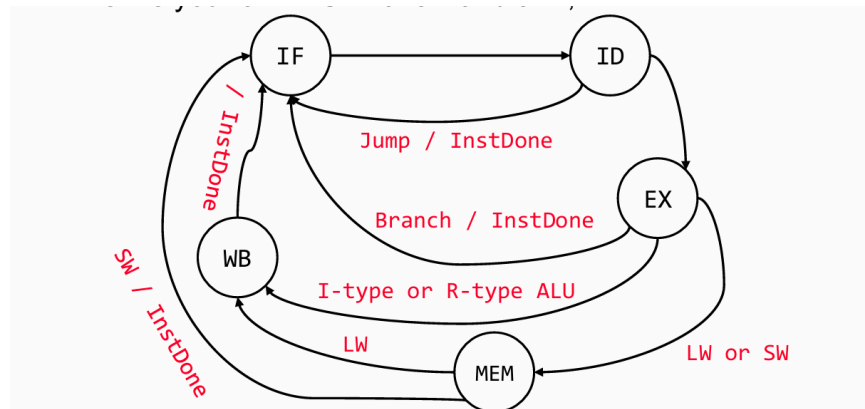


lab4_report

0. 목표

Multicycle CPU만들기

1. control signal 추가하기.
2. FSM 설계하기



InstDone 신호 대신 그냥 IF단계로 상태를 변경하는 것으로 구현할 예정.

jal : IF → ID → EX → WB

jr : IF → ID → EX

3. datapath 다시 설계하기
4. resource 공유하기
 - a. 구현을 위해 uarchitecture 필요
 - b. PC계산도 이제 ALU를 이용해야 함.

1. CTRL.cpp

구현 계획

이전 STATE와 instruction을 가지고 다음 수행할 단계의 signal 생성.

최대한 제공된 그래프에 나오는 signal로 모든 것을 표현하려고 노력했다.

추가적으로 사용한 signal로는

`SignExtend` 와, `ra` 에 WB하기 위한 `RegDst` 의 11 비트, `jr`에서 31번 레지스터를 지정하기 위해 사용한 `PCSource` 의 11 비트가 있다.

기능 구현

우선, 새로 추가된 신호에 맞춰서, `signal`을 생성했다.

위 첨부된 FSM을 따라 `state`를 변경해가며 작동할 수 있도록 구현했다.

우선, 매 cycle마다 `controlSignal` 함수를 호출할 때, `control bit`들은 초기화된다.

```
... *controls = {}; // 모든 제어 신호 초기화
```

IF, ID의 경우 공통적으로 모든 명령어가 수행하기 때문에 동일하게 구현했다.

```
// 공통 단계들의 control signal 설정
if (state == InstFetch) {
    // IF 단계 공통 신호 (모든 명령어 동일)
    controls->MemRead = 1;    // 메모리 읽기
    controls->IRWrite = 1;    // IR에 쓰기 허용
    controls->ALUSrcA = 0;    // PC를 ALU 입력으로
    controls->ALUSrcB = 1;    // 4를 ALU 입력으로 (PC+4)
    controls->ALUOp = ALU_ADDU; // ADD 연산 (PC+4)
    controls->PCWrite = 1;    // PC 업데이트
    controls->PCSource = 0;   // PC+4
    controls->IorD = 0;      // PC를 주소로 사용
    state = InstDecode;
    return;
}

if (state == InstDecode) {
    // ID 단계 공통 신호 (모든 명령어 동일)
    controls->ALUSrcA = 0;    // PC를 ALU 입력으로
    controls->ALUSrcB = 3;    // sign-extended imm << 2를 ALU 입력으로
    controls->ALUOp = ALU_ADDU; // ADD 연산 (PC + offset)
    // 다음 상태를 opcode에 따라 분기
    switch (Inst->opcode) {
        case OP_RTYPE:
            if (Inst->funct == FUNCT_JR) state = Execute;
            else state = Execute;
            break;
        case OP_LW: case OP_SW: state = Execute; break;
        case OP_BEQ: case OP_BNE: state = Execute; break;
        case OP_J: state = Execute; break;
        case OP_JAL: state = Execute; break;
        default: state = Execute; break;
    }
    return;
}
```

이후 EX단계부터는, 각 명령어에 맞춰, `signal`을 만들어줬고, 다음 `state`를 지정했다.
R타입의 경우만 사진으로 첨부하겠다.

```

// EX, MEM, WB 단계는 명령어 타입에 따라 다르게 처리
if (Inst->opcode == OP_RTYPE) {
    switch(state) {
        case Execute:
            controls->ALUSrcA = 1; // A
            controls->ALUSrcB = 0; // B

            if (Inst->funct == FUNCT_JR) {
                controls->PCSource = 3; // jr
                controls->PCWrite = 1; // PC 업데이트
                state = InstFetch;
            } else {
                switch(Inst->funct) {
                    case FUNCT_SLL: controls->ALUOp = ALU_SLL; break;
                    case FUNCT_SRL: controls->ALUOp = ALU_SRL; break;
                    case FUNCT_SRA: controls->ALUOp = ALU_SRA; break;
                    case FUNCT_ADDU: controls->ALUOp = ALU_ADDU; break;
                    case FUNCT_SUBU: controls->ALUOp = ALU_SUBU; break;
                    case FUNCT_AND: controls->ALUOp = ALU_AND; break;
                    case FUNCT_OR: controls->ALUOp = ALU_OR; break;
                    case FUNCT_XOR: controls->ALUOp = ALU_XOR; break;
                    case FUNCT_NOR: controls->ALUOp = ALU_NOR; break;
                    case FUNCT_SLT: controls->ALUOp = ALU_SLT; break;
                    case FUNCT_SLTU: controls->ALUOp = ALU_SLTU; break;
                }
                state = WriteBack;
            }
            break;

        case WriteBack:
            if (Inst->funct != FUNCT_JR) {
                controls->RegWrite = 1;
                controls->RegDst = 1; // rd
                controls->MemtoReg = 0; // ALUOut
                state = InstFetch;
            }
            break;
    }
}

```

보면, Execute 단계에서 모든 연산에 대한 ALUOp를 지정해줬다. 또한 각 연산들은 ALUSrc로 A와 B에 저장된 값을 사용하므로, 이에 맞춰 설정했다. 이후, WriteBack 단계로 넘어간다.

jr의 경우, 31번 레지스터에서 값을 가져오기 위해 새로 지정한

PCSource의 11비트를 이용하도록 했고, 이를 이용해 PC값을 업데이트 하도록 설정했다.

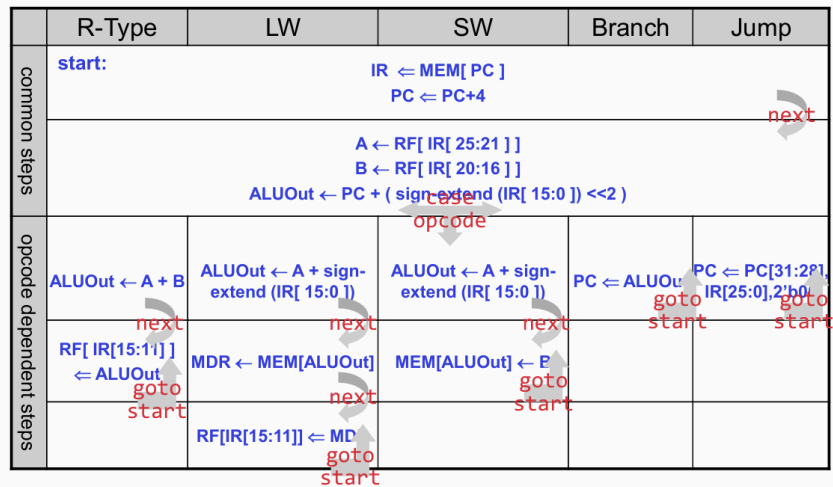
jr은 해당 연산이 명령어의 끝이므로, 업데이트된 PC값이 가리키는 명령어로 InstFetch를 진행하도록 다음 state를 설정했다.

구현하고 보니, Inst를 통째로 받지 않고, HW3 controlSignal 함수처럼 opcode와 funct만 받아서 사용해도 문제 없었을 것으로 보인다.

2. CPU.cpp

구현 계획

Combined RT Sequencing



jal의 경우, opcode dependent steps에

ex단계 ~~$ALUOut \leftarrow PC + 4$~~

ex단계 $PC \leftarrow PC[31:28], IR[25:0], 2'b00$

wb단계 $RF[31] \leftarrow ALUOut$

jr의 경우, opcode dependent steps에

ex단계 PC ra에서 읽어와서 거기서 jump

STATE가 IF인 경우에 IF를 진행하면, Fetch한 정보는 Instruction register에 남아있으니, 이를 이용한다.

하단에서 현재 STATE와 instruction으로 signal 생성한다.

코드 중간 부분에서는, 결정된 signal에 따라 코드 수행한다.

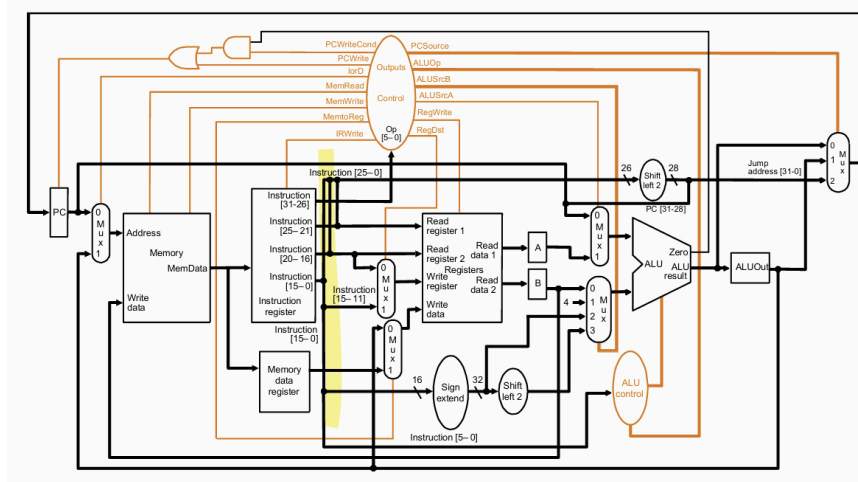
추가적으로,

중간에 존재하는 microarchitectural state를 활용해야 하고,

모든 연산들에는 이제 alu를 사용한다.

(수정) 처음에는, CPU.cpp에서 CurrentState라는 변수를 만들어서 한 단계씩 구현을 하고 있었는데, 자원을 공유하게 하지 못하고 있다는 생각이 들었다.

또한, multicycle의 취지에 맞게 만들기 위해서는, 각 함수들의 호출이 한번만 이루어져야 하는 것이 아닐까 하는 근본적인 의문점이 들어, 다시 구현하게 되었다.



우선, 함수를 호출하기 전에 MUX에서 어떤 값을 사용할 지 정하는 부분들을 모두 고려해주면 단 한번만 호출이 가능할 것이라고 생각하여, 해당 그래프를 그대로 코드로 변환하였다.

jr 구현의 경우,

jr 명령어는 레지스터 rs에 저장된 값을 바로 PC로 복사해서 점프하는 명령어이기 때문에, rs에서 읽은 값이 저장된 A를 사용했다.

A를 가져가기 위한 신호를 만드는 데에는, PCSource의 11이 사용되지 않고 있기 때문에, 해당 값으로 jr을 표현했다.

jal 구현의 경우,

ra에 값을 적는 경우, WB하는 RegDst값으로 2를 사용했다. (RegDst를 2bit로 변화시켜 사용했다.)

기능 구현

위 Datapath와 최대한 동일하게 구현을 진행했다.

처음에는 microarchitecture에 저장한 값을 사용하는 관계를 생각하며 Datapath에 그려진 흐름(왼쪽 → 오른쪽)대로 코드가 작성되진 않았었는데,

모든 오류를 해결하고 나니, 순서대로 배치하여도 정상적으로 작동했다.

microarchitecture에 저장된 값들은 다음 cycle에서 사용되기 때문이다.

```
ctrl.splitInst(IR, &Inst);
ctrl.controlSignal(currentState, &Inst, &controls);
```

state를 통해 signal 및 다음 state 설정

이후 호출되는 모든 구간의 함수들은, 내부적으로 controls에 담겨있는 신호를 기반으로, 이번 cycle에 사용될 지, 사용되지 않을 지 자체적으로 결정된다.

예를 들어, rf.write를 생각해보면, controls.RegWrite 신호가 0이라면, 해당 함수를 호출했지만, 아무 일도 일어나지 않고 그대로 빠져나온다. 즉, WB stage를 하지 않고 지나갔다는 의미이다.

이런 식으로, signal을 기반으로 매 cycle마다 한 단계씩 진행을 하도록 구현했다.

```
// IF, MEM
addr = (controls.IorD) ? ALUOut : PC;
mem.memAccess(addr, &mem_data, B, controls.MemRead, controls.MemWrite);
if(mem_data == 0) { // IF 단계에서 읽어올 명령어가 없으면 종료
    status = TERMINATE;
    return 0;
}
```

```
// ID
// printf("rs: %d, rt: %d, rd: %d\n", Inst.rs, Inst.rt, Inst.rd);
rf.read(Inst.rs, Inst.rt, &rs_data, &rt_data);
ctrl.signExtend(Inst.immi, controls.SignExtend, &ext_imm);
```

```
// EX
operand1 = (controls.ALUSrcA) ? A : PC;
operand2 = 0; //초기화
if(controls.ALUSrcB == 0) operand2 = B;
else if(controls.ALUSrcB == 1) operand2 = 4; //PC+4
else if(controls.ALUSrcB == 2) operand2 = ext_imm; //sign-extended imm
else if(controls.ALUSrcB == 3) operand2 = ext_imm << 2; //sign-extended imm
alu.compute(operand1, operand2, Inst.shamt, controls.ALUOp, &alu_result);
```

```
// WB
if(controls.RegDst == 0) wr_addr = Inst.rt;
else if(controls.RegDst == 1) wr_addr = Inst.rd;
else if(controls.RegDst == 2) wr_addr = 31; // $ra
else wr_addr = 0; // default case - 어차피 안에서 write를 안함.
if(controls.MemtoReg == 0) wr_data = ALUOut;
else if(controls.MemtoReg == 1) wr_data = MDR;
else wr_data = 0; // default case - 문제 없으려나..?
rf.write(wr_addr, wr_data, controls.RegWrite);
```

PC값의 경우, 어떤 함수를 호출하여 작업하는 것이 아니기 때문에, 해당 cycle에서 생성된 signal 기반으로 PC값을 업데이트 할 수 있도록 분기문을 작성하여 구현했다.

```
// Update the PC
if ((controls.PCWriteCond && alu_result == 1) || controls.PCWrite) {
    switch (controls.PCSource) {
        case 0: PC = alu_result; break; // 일반적 PC+4
        case 1: PC = ALUOut; break; // branch target
        case 2: PC = (PC & 0xF0000000) | (Inst.immj << 2); break; // jump
        case 3: PC = A; break; // jr
    }
}
```

코드 최하단에서, microarchitecture를 업데이트해줬다.
해당 값들은 다음 cycle에서 사용될 것이다.

```
if(controls.IRWrite) IR = mem_data; // IR에 명령어 저장
else MDR = mem_data;
ALUOut = alu_result;
A = rs_data;
B = rt_data;
// printf("PC: %08x, IR: %08x, A: %08x, B: %08x, ALUOut: %08x\n", PC, IR, A, B, ALUOut);
return 1;
```

3. Test (cpp)

testcase1 - 실패 → 성공

```
$zero 00000000
$at    fffffa6e4
$v0    00000000
$v1    00000000
$a0    00000000
$a1    00000000
$a2    00000000
$a3    00000000
$t0    000001db
$t1    00000001
$t2    fffffa6e5
$t3    00000001
$t4    ffff4a25
$t5    00000000
$t6    00000000
$t7    00000000
$s0    ffffffff
$s1    fffffba3a
$s2    00000001
$s3    ffff4221
$s4    00000000
$s5    00000001
$s6    00000000
$s7    00000000
$t8    00000001
$t9    00000596
$k0    00000000
$k1    00000000
$gp    00001800
$sp    00003ffc
$fp    00000000
$ra    00000000
```

```
0x00000000
0xfffffa6e4
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x000001db
0x00000001
0xfffffa6e5
0x00000001
0xffff4a25
0x00000000
0x00000000
0x00000000
0x00000000
0xffffffff
0xfffffba3a
0x00000001
0xffff4221
0x00000000
0x00000001
0x00000000
0x00000000
0x00000001
0x00000596
0x00000000
0x00000000
0x00001800
0x00003ffc
0x00000000
0x00000000
0x000009b8
0x00000000
0x00000000
```

testcase2 - 실패 → 실패 → 성공

\$zero	00000000
\$at	fff90000
\$v0	00000005
\$v1	00000000
\$a0	00000000
\$a1	00000000
\$a2	00000000
\$a3	00000000
\$t0	fff8e8df
\$t1	fff9d354
\$t2	fff8e8df
\$t3	fff8e8df
\$t4	ffea7884
\$t5	fff9f0cb
\$t6	00000000
\$t7	00000000
\$s0	fff8e8df
\$s1	fff8e8df
\$s2	fff8e8df
\$s3	fff9d354
\$s4	ffea7884
\$s5	00042808
\$s6	00000000
\$s7	00000000
\$t8	00000000
\$t9	00000000
\$k0	00000000
\$k1	00000000
\$gp	00001800
\$sp	00003ffc
\$fp	00000000
\$ra	00000000

[illegible]

testcase3 - 성공

\$zero	00000000
\$at	00000000
\$v0	00000028
\$v1	00000000
\$a0	00000000
\$a1	00000000
\$a2	00000000
\$a3	00000000
\$t0	0000003c
\$t1	00000028
\$t2	00000028
\$t3	00000000
\$t4	00000000
\$t5	00000000
\$t6	00000000
\$t7	00000000
\$s0	00000000
\$s1	00000000
\$s2	00000000
\$s3	00000000
\$s4	00000000
\$s5	00000000
\$s6	00000000
\$s7	00000000
\$t8	00000000
\$t9	00000000
\$k0	00000000
\$k1	00000000
\$gp	00001800
\$sp	00003ffc
\$fp	00000000
\$ra	00000000

[illegible]

testcase4 - 성공

[illegible]

```
$zero 00000000
$at    22400000
$v0    00000005
$v1    00000000
$a0     00000000
$a1     00000000
$a2     00000000
$a3     00000000
$t0     74e250d2
$t1     fbd5181f
$t2     a1fccadd
$t3     5efc37c0
$t4     63f89338
$t5     e6442b6e
$t6     00000000
$t7     00000000
$w0     e219c8e6
$w1     11bbba2a
$w2     65a429ac
$w3     13d0e99e
$w4     61116734
$w5     fdeaf088
$w6     00000000
$w7     00000000
$w8     00000000
$w9     00000000
$w0     00000000
$w1     00000000
$w2     00001800
$w3     00003ffc
$w4     00000000
$w5     00000034
```

0x00000000
0x224d0000
0x00000005
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x74e250d2
0xfdb5a118
0xa16cc4d0
0x5efc37c0
0x638f9338
0xe6442b6e
0x00000000
0x00000000
0xe219cc86
0x11bba2a
0x65a42cac
0x13d0e99e
0xf11f6734
0x861ead08c
0x00000000
0x00000000
0x00000000
0x00000000
0x00001800
0x00003ffc
0x00000000
0x00000034

```

$zero 00000000
$at   aa210000
$v0   00000005
$v1   00000000
$a0   00000000
$a1   00000000
$a2   00000000
$a3   00000000
$t0   d7c716b4
$t1   d7c716b4
$t2   00000000
$t3   00000000
$t4   00003457
$t5   00000000
$t6   00000000
$t7   00000000
$s0   00000000
$s1   00000000
$s2   00000000
$s3   00000000
$s4   00000000
$s5   00000000
$s6   00000000
$s7   00000000
$t8   00000000
$t9   00000000
$k0   00000000
$k1   00000000
$gp   00001800
$sp   00003ffc
$fp   00000000
$ra   00000000

```

```

0x00000000
0xaa210000
0x00000005
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0xd7c716b4
0xd7c716b4
0x00000000
0x00000000
0x00003457
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00001800
0x00003ffc
0x00000000
0x00000000

```

testcase7 - 성공

```

$zero 00000000
$at   e2810000
$v0   00000000
$v1   00000000
$a0   00000000
$a1   00000000
$a2   00000000
$a3   00000000
$t0   ffffffff
$t1   00000000
$t2   ffffffff
$t3   ffffffff
$t4   00000000
$t5   00000000
$t6   00000000
$t7   00000000
$s0   00000000
$s1   00000000
$s2   00000000
$s3   00000000
$s4   00000000
$s5   00000000
$s6   00000000
$s7   00000000
$t8   00000000
$t9   00000000
$k0   00000000
$k1   00000000
$gp   00001800
$sp   00003ffc
$fp   00000000
$ra   00000000

```

```

0x00000000
0xe2810000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0xffffffff
0x00000000
0xffffffff
0xffffffff
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00000000
0x00001800
0x00003ffc
0x00000000
0x00000000

```

4. Trouble Shooting (cpp)

testcase1

4.1. 코드 순서 변경

Execute 단계에서 연산이 완료된 값이 RF에 적히기 전에, PC 연산에 의해 덮여버리는 문제가 있었다.

Execute 연산의 결과가 저장되는 `ALUOut` 속 값은 다음 cycle에서 사용하기 때문에, 코드의 맨 마지막에서 `ALUOut`에 넣어주면 해결될 것이라 생각하고, 코드를 수정했다.

```
}  
  
ALUOut = alu_result;  
// for debugging  
printf("PC: %08x, IR: %08x, A: %08x, B: %08x", PC, IR, A, B);  
return 1;  
}
```

이후, 테스트를 다시 진행하니 정상적으로 결과가 나왔다.

4.2. Out of bound data memory access

확인해보니, 메모리에서 IF를 했을 때, 읽어온 명령어가 `00000000`인 경우, 즉, 마지막 명령어까지 모두 수행한 이후 해당 코드가 출력되는 것을 확인했다.

```
Out of bound data memory access  
RF states after the program execution
```

메모리 접근 이후, `mem_data`를 검증하는 과정을 추가하여 이를 해결했다.

```
mem.memAccess(addr, &mem_data, B, controls.MemRead, controls.MemWrite);  
if(mem_data == 0){ // IF 단계에서 읽어올 명령어가 없으면 종료  
    status = TERMINATE;  
    return 0;  
}
```

```
Simulation done successfully.  
RF states after the program execution
```

testcase2

4.3 `addu $20, $20, $17`

```

PC: 00000090, IR: 00007fb0, A: 00000000, B: 00074a13, ALUOut: 00074aa3, MDR: 00007fb0
$zero 00000000
$at    fff30000
$v0    00000000
$v1    00000000
$a0    00000000
$a1    00000000
$a2    00000000
$a3    00000000
$t0    fff746f4
$t1    fff9c2d1
$t2    0006f3c6
$t3    fff1829b
$t4    fff746f4
$t5    00000000
$t6    00000000
$t7    00000000
$s0    00074a13
$s1    fff746f4
$s2    0003512f
$s3    fff3a9db
$s4    00074a13
$s5    00000000
$s6    00000000
$s7    00000000
$t8    00000000
$t9    00000000
$k0    00000000
$k1    00000000
$gp    00001800
$sp    00003ffc
$fp    00000000
$ra    00000000

```

0x00000098	0x0291a021	addu \$20,\$20,\$17	36:	addu \$s4, \$s4, \$s1
------------	------------	---------------------	-----	-----------------------

4.4 beq \$8, \$16, 0x00000010

계속해서 문제가 해결되지 않아, 혹시나 하고 한단계씩 돌려보니, 진짜 문제는 위의 addu 연산이 아니었다..

그 직전에 있던 beq가 문제였다. branch가 발생하지 않아야하는데 branch가 발생했다. 실제로는 위의 addu가 있는 위치에 가지 않아야했다.

00040	0x00114021	addu \$8,\$0,\$17	12:	move \$t0, \$s1
00044	0x11100010	beq \$8,\$16,0x00000010	13:	beq \$t0, \$s0, eq_0_0_0
00048	0x11110012	beq \$8,\$17,0x00000012	14:	beq \$t0, \$s1, eq_0_0_1
0004c	0x11120014	beq \$8,\$18,0x00000014	15:	beq \$t0, \$s2, eq_0_0_2

```

Current State: Execute
rs: 8, rt: 16, rd: 0
1-A: fff746f4, B: 00074a13, ALUOut: 00000088
2-A: fff746f4, B: 00074a13, ALUOut: 00000088
PC: 00000088, IR: 00007fff, A: fff746f4, B: 00074a13
$zero 00000000

```

```

/ Update the PC
f ((controls.PCWriteCond && alu_result == 0) || controls.PCWrite) {
  switch (controls.PCSource) {
    case 0: PC = alu_result; break; // 일반적 PC+4
    case 1: PC = ALUOut; break; // branch target
    case 2: PC = (PC & 0xF0000000) | (Inst.immj << 2); break; // jump
    case 3: PC = A; break; // jr
  }
}

```

아아.. EQ연산을 했을 때 같으면 1을 반환하기 때문에 alu_result == 1을 해야하는데, 'Zero'라는 글씨를 보고 착각을 했었나보다.

4.5 j 0x00000148

점프를 뛰어야 하는데, 이상한 곳으로 점프를 뛰었다.

0x0000009c	0x08000052	j 0x00000148
0x000000a0	0x01886021	addu \$12,\$12,\$8

```
// 다음 상태를 opcode에 따라 분기
switch (inst->opcode) {
    case OP_RTYPE:
        if (inst->funct == FUNCT_JR) state = InstFetch;
        else state = Execute;
        break;
    case OP_LW: case OP_SW: state = Execute; break;
    case OP_BEQ: case OP_BNE: state = Execute; break;
    case OP_J: state = InstFetch; break;
    case OP_JAL: state = Execute; break;
    default: state = Execute; break;
}
return;
```

확인해보니, 다음 state가 Execute가 아닌 InstFetch로 되어있었다.
해당 코드를 수정하니, testcase2.hex를 정상적으로 통과했다.

testcase5

4.6 jal 계산 문제

jal의 최종 목적지와 ra에 저장되는 주소값이 모두 4씩 더 증가되어 있었다.

```
155 }
156 else if (inst->opcode == OP_JAL) {
157     switch (state) {
158         case Execute:
159             // controls->ALUSrcA = 0; // PC
160             // controls->ALUSrcB = 1; // 4
161             // controls->ALUOp = ALU_ADDU; // PC + 4
162             controls->PCWrite = 1;
163             controls->PCSource = 2; // jump address
164             state = WriteBack; // $ra에 PC+4 저장을 위해
165             break;
```

pc+4를 두 번 계산하고 있었다..
IF에서 이미 한 번 진행 했었는데
이를 지워 해결했다.

4.7 jr 작동 문제

ra레지스터에 저장된 값으로 jump하지 않고 그대로 PC+4를 하는 것을 확인할 수 있었다.
확인해보니, decoding까지만 진행하고, 이전엔 InstFetch로 넘어가도록 설정했었다.
jr은 alu를 사용해서 연산을 하지 않고, 그냥 ra에 있는 곳으로 점프를 뛰니, ex 단계가 필요

없다고 생각했었던 것 같다.

하지만, Execute 단계를 추가하여, PC를 업데이트를 하는 과정을 거쳐줘야 했다.

```
if (inst->func == FUNCT_JR) {  
    controls->PCSource = 3; // jr  
    controls->PCWrite = 1;  // PC 업데이트  
    state = InstFetch;  
}
```

놀랍게도, 해당 코드는 이미 만들어졌었다.
InstFetch 단계로 바로 넘어가서 사용하지 못했을 뿐.

```
case OP_RTYPE:  
    if (inst->func == FUNCT_JR) state = Execute;  
    else state = Fetch;
```

Execute 단계로 가도록 설정하여 해결했다.

코드 간소화

cpp를 구현하면서 CTRL.cpp 속 함수 controlSignal()에 받는 인자를 CPUState와 ParsedInst형의 값을 가져왔는데, 이 중 ParsedInst의 경우, singlecycle cpu에서 구현했을 때와 마찬가지로, opcode와 func밖에 이용하지 않았다.

즉, CPUState만 추가했어도 문제가 없었을 것이다.

하지만, 구현 상 문제가 없기 때문에 해당 내용은 굳이 수정하진 않고,
해당 코드를 간소화할 수 있다는 사실을 언급하는 정도로 넘어가겠다.

0. 대략적인 verilog 전체 구현 계획

tick마다 진행해야 할 일.

signal 새로 생성, state 업데이트

microarchitecture에 값 업데이트

1. CTRL.v

구현 계획

cpp과 동일하게 구현할 예정이다.

다만, 몇몇 signal들은 사용하는 bit수가 다르기 때문에, 선언 과정에서 유의해야 한다.

그 외에는 switch-case를 이용하여 매 cycle마다 사용할 signal을 적절히 분배해주고, 다음 state를 지정해주면 된다.

추가적으로, 혹시나 동기화 문제가 생길까 싶어, state를 이번에는 모듈 내부에서 바로 변경하는 것이 아니라,

다음 state를 output으로 설정해두고,

값을 갱신하는 것은

CPU.v의 clock에 맞춰 할 예정이다.

기능 구현

우선 모듈에서 사용할 변수들은 다음과 같이 bit수를 고려하여 정의했다.

```
module CTRL(  
    // input opcode and funct  
    input [5:0] opcode,  
    input [5:0] funct,  
    input [2:0] state,
```

```
// output various ports
output reg PCWrite,
output reg PCWriteCond,
output reg MemRead,
output reg MemWrite,
output reg IorD,
output reg IRWrite,
output reg [1:0] RegDst,
output reg RegWrite,
output reg MemtoReg,
output reg ALUSrcA,
output reg [1:0] ALUSrcB,
output reg [3:0] ALUOp,
output reg [1:0] PCSource,
output reg SignExtend,
output reg [2:0] nextState
);
```

추가적으로 `state`, `nextState` 에 대한 값 지정을 수월하게 하기 위해 지역 변수를 선언했다.

```
//CPUState 정의
localparam InstFetch = 3'd0;
localparam InstDecode = 3'd1;
localparam Execute = 3'd2;
localparam Memory = 3'd3;
localparam WriteBack = 3'd4;
```

signal을 생성하는 부분을 살펴보자.

매번 signal을 생성할 때마다, 값을 할당하기 이전에 초기화해주는 작업을 추가했다.


```

always @(*) begin
    PCWrite = 0;
    PCWriteCond = 0;
    MemRead = 0;
    MemWrite = 0;
    MemtoReg = 0;
    IorD = 0;
    IRWrite = 0;
    RegDst = 2'd0;
    RegWrite = 0;
    ALUSrcA = 0;
    ALUSrcB = 2'd0;
    ALUOp = 4'd0;
    PCSource = 2'd0;
    SignExtend = 0;
    nextState = state;
end

```

값을 할당하는 부분은 cpp 구현과 마찬가지로 switch구문을 사용하여 적절히 signal을 할당했다.

```

case (state)
    InstFetch: begin

```

```

    InstDecode: begin

```

```

    Execute: begin

```

```

    Memory: begin

```

```

    WriteBack: begin

```

2. CPU.v

구현 계획

우선, '저장'의 기능을 담당할 microarchitectural state와, PC, 그리고 currentState만 reg로 선언하고,

그 외의 signal들이나 모듈과 연결에 사용할 여러 값들은 wire로 선언할 예정이다.

그런 다음, clock이 될 때마다, reg로 선언한 변수들에 값을 저장해주는 느낌으로 구현하면 될 것 같다.

그 외의 값들은 always구문 밖에서 assign으로 적절히 연결해줄 예정이다.

signal에 따라서 alu 연산에 어떤 값을 사용할 것인지만,

rf할 때 어떤 주소를 사용할 것인지 등을 지정해주면 될 것으로 보인다.

기능 구현

우선, '저장'을 할 microarchitectural state들을 reg로 선언해줬다.

```
// 주요 레지스터들
reg [31:0] PC;
reg [31:0] IR;
reg [31:0] MDR;
reg [31:0] A;
reg [31:0] B;
reg [31:0] ALUOut;
```

그 외에 모듈과 연결할 wire들을 선언해주고

```
//wire [31:0] inst;
wire [31:0] mem_data;
wire [31:0] mem_addr;
wire [31:0] alu_result;
wire [31:0] ext_imm;

wire [31:0] rd_data1;
wire [31:0] rd_data2;
wire [4:0] wr_addr;
wire [31:0] wr_data;
```

IR에 저장된 명령어들을 나눠줬다.

```
// Instruction fields
wire [5:0] opcode = IR[31:26];
wire [4:0] rs = IR[25:21];
wire [4:0] rt = IR[20:16];
wire [4:0] rd = IR[15:11];
wire [4:0] shamt = IR[10:6];
wire [5:0] funct = IR[5:0];
wire [15:0] immi = IR[15:0];
wire [25:0] immj = IR[25:0];
```

마지막으로, signal 값을 받아올 wire를 설정해줬다.

단, 여기도 CTRL.v와 마찬가지로, bit수가 달라진 signal들을 주의해야 했다.
(RegDst는 처음에 잘못 선언해두어서 testcase5에서 문제가 발생했었다.)

```
// Control signals
wire          PCWrite;
wire          PCWriteCond;
wire          MemRead;
wire          MemWrite;
wire          IorD;
wire          IRWrite;
wire [1:0]    RegDst;
wire          RegWrite;
wire          MemtoReg;
wire          ALUSrcA;
wire [1:0]    ALUSrcB;
wire [3:0]    ALUOp;
wire [1:0]    PCSource;
wire          SignExtend;
wire [2:0]    nextState;
```

우선, clock이 될 때마다 '저장'할 값들을 바꿔주는 부분은 다음과 같이 구현했다.
PC, IR, MDR, A, B, ALUOut, currentState에 값을 넣어주되, 고려해야 할 signal들을 기준으로 진행했다.

```
if (rst) begin //동기적으로 rst
    PC <= 0;
    IR <= 0;
    MDR <= 0;
    A <= 0;
    B <= 0;
    ALUOut <= 0;
    currentState <= InstFetch;
end else begin
    if (IRWrite) IR <= mem_data;
    else MDR <= mem_data;
    if ((PCWriteCond && alu_result == 1) || PCWrite) begin
        case (PCSource)
            2'd0: PC <= alu_result;
            2'd1: PC <= ALUOut;
            2'd2: PC <= {PC[31:28], immj, 2'b00};
            2'd3: PC <= A; //JR
        endcase
    end
    A <= rd_data1;
    B <= rd_data2;
    ALUOut <= alu_result;
    currentState <= nextState;
end
end
```

그 외에, 모듈에 넣어줄 값을 정해주는 부분도 signal을 기준으로 진행했다.

```

// Sign Extension
assign ext_imm = SignExtend ? {{16{immi[15]}}, immi} : {16'b0, immi};
// MEM
assign mem_addr = (IorD) ? ALUOut : PC;
// WB
assign wr_addr = (RegDst == 2'd0) ? rt :
                 (RegDst == 2'd1) ? rd :
                 (RegDst == 2'd2) ? 5'd31 : // JAL
                 5'd0; // default
assign wr_data = (MemtoReg == 1'b0) ? ALUOut : MDR;

```

ALU 모듈과의 연결하는 부분은, 연결 선언을 하는 부분에서 분기문을 사용했다.

그 외 모듈과의 연결은, 각 verilog 파일에서 선언된 대로 적절하게 배치해주었다.

몇몇 signal을 추가하지 않아, 후에 문제가 발생하기도 했었다.

```

//연결
CTRL ctrl (
    .opcode(opcode),
    .funct(funct),
    .state(currentState),
    .PCWrite(PCWrite),
    .PCWriteCond(PCWriteCond),
    .MemRead(MemRead),
    .MemWrite(MemWrite),
    .IorD(IorD),
    .IRWrite(IRWrite),
    .RegDst(RegDst),
    .RegWrite(RegWrite),
    .MemtoReg(MemtoReg),
    .ALUSrcA(ALUSrcA),
    .ALUSrcB(ALUSrcB),
    .ALUOp(ALUOp),
    .PCSource(PCSource),
    .SignExtend(SignExtend),
    .nextState(nextState)
);

```

```

RF rf (
    .clk(clk), .rst(rst),
    .rd_addr1(rs), .rd_addr2(rt),
    .rd_data1(rd_data1), .rd_data2(rd_data2),
    .RegWrite(RegWrite),
    .wr_addr(wr_addr), .wr_data(wr_data)
);

```

```

MEM mem (
    .clk(clk), .rst(rst),
    .mem_addr(mem_addr), .MemWrite(MemWrite),
    .mem_write_data(B), .mem_read_data(mem_data)
);

```

```

ALU alu (
    .operand1(ALUSrcA ? A : PC),
    .operand2((ALUSrcB == 2'd0) ? B :
              (ALUSrcB == 2'd1) ? 32'd4 :
              (ALUSrcB == 2'd2) ? ext_imm :
              (ext_imm << 2)),
    .shamt(shamt),
    .funct(ALUOp),
    .alu_result(alu_result)
);

```

3. Test (v)

testcase1 - 성공

> [0][31:0]	00000000	havruta@BOOK-TCTPS.JLV
> [1][31:0]	ffffa6e4	u testcase1.hex
> [2][31:0]	00000000	Loading instruction me
> [3][31:0]	00000000	c
> [4][31:0]	00000000	Starting CPU simulatio
> [5][31:0]	00000000	Simulation done succes
> [6][31:0]	00000000	RF states after the pr
> [7][31:0]	00000000	\$zero 00000000
> [8][31:0]	00000000	\$at fffffa6e4
> [9][31:0]	00000000	\$v0 00000000
> [10][31:0]	000001db	\$v1 00000000
> [11][31:0]	00000001	\$a0 00000000
> [12][31:0]	ffffa6e5	\$a1 00000000
> [13][31:0]	00000001	\$a2 00000000
> [14][31:0]	ffff4a25	\$a3 00000000
> [15][31:0]	00000000	\$t0 000001db
> [16][31:0]	00000000	\$t1 00000001
> [17][31:0]	00000000	\$t2 fffffa6e5
> [18][31:0]	00000000	\$t3 00000001
> [19][31:0]	00000000	\$t4 fffff4a25
> [20][31:0]	ffffffe	\$t5 00000000
> [21][31:0]	ffffba3a	\$t6 00000000
> [22][31:0]	00000001	\$t7 00000000
> [23][31:0]	ffff4221	\$s0 ffffffffefe
> [24][31:0]	00000000	\$s1 fffffba3a
> [25][31:0]	00000001	\$s2 00000001
> [26][31:0]	00000000	\$s3 fffff4221
> [27][31:0]	00000000	\$s4 00000000
> [28][31:0]	00000000	\$s5 00000001
> [29][31:0]	00000000	\$s6 00000000
> [30][31:0]	00000001	\$s7 00000000
> [31][31:0]	00000596	\$t8 00000001
> [32][31:0]	00000000	\$t9 00000596
> [33][31:0]	00000000	\$k0 00000000
> [34][31:0]	00000000	\$k1 00000000
> [35][31:0]	00001800	\$gp 00001800
> [36][31:0]	00003ffc	\$sp 00003ffc
> [37][31:0]	00000000	\$fp 00000000
> [38][31:0]	00000000	\$ra 00000000

Simulation success!!!

testcase2 - 실패 → 성공

> [0][31:0]	00000000	havruta@BOOK-TCTPS.JLV
> [1][31:0]	fff90000	u testcase2.hex
> [2][31:0]	00000005	Loading instruction me
> [3][31:0]	00000000	c
> [4][31:0]	00000000	Starting CPU simulatio
> [5][31:0]	00000000	Simulation done succes
> [6][31:0]	00000000	RF states after the pr
> [7][31:0]	00000000	\$zero 00000000
> [8][31:0]	00000000	\$at fff90000
> [9][31:0]	00000000	\$v0 00000005
> [10][31:0]	fff8e8df	\$v1 00000000
> [11][31:0]	fff9d354	\$a0 00000000
> [12][31:0]	fff8e8df	\$a1 00000000
> [13][31:0]	fff8e8df	\$a2 00000000
> [14][31:0]	fff8e8df	\$a3 00000000
> [15][31:0]	ffea7884	\$t0 fff8e8df
> [16][31:0]	fff9f0cb	\$t1 fff9d354
> [17][31:0]	00000000	\$t2 fff8e8df
> [18][31:0]	00000000	\$t3 fff8e8df
> [19][31:0]	00000000	\$t4 ffea7884
> [20][31:0]	fff8e8df	\$t5 fff9f0cb
> [21][31:0]	fff8e8df	\$t6 00000000
> [22][31:0]	fff8e8df	\$t7 00000000
> [23][31:0]	fff9d354	\$s0 fff8e8df
> [24][31:0]	ffea7884	\$s1 fff8e8df
> [25][31:0]	00042808	\$s2 fff8e8df
> [26][31:0]	00000000	\$s3 fff9d354
> [27][31:0]	00000000	\$s4 ffea7884
> [28][31:0]	00000000	\$s5 00042808
> [29][31:0]	00000000	\$s6 00000000
> [30][31:0]	00000000	\$s7 00000000
> [31][31:0]	00000000	\$t8 00000000
> [32][31:0]	00000000	\$t9 00000000
> [33][31:0]	00000000	\$k0 00000000
> [34][31:0]	00000000	\$k1 00000000
> [35][31:0]	00001800	\$gp 00001800
> [36][31:0]	00003ffc	\$sp 00003ffc
> [37][31:0]	00000000	\$fp 00000000
> [38][31:0]	00000000	\$ra 00000000

Simulation success!!!

testcase3 - 성공

> [0][31:0]	00000000	havruta@BOOK-TCTPSJLV91
> [1][31:0]	00000000	u testcase3.hex
> [2][31:0]	00000028	Loading instruction mem
> [3][31:0]	00000000	c
> [4][31:0]	00000000	Starting CPU simulation
> [5][31:0]	00000000	Simulation done success
> [6][31:0]	00000000	RF states after the pro
> [7][31:0]	00000000	\$zero 00000000
> [8][31:0]	0000030c	\$at 00000000
> [9][31:0]	00000028	\$v0 00000028
> [10][31:0]	00000028	\$v1 00000000
> [11][31:0]	00000000	\$a0 00000000
> [12][31:0]	00000000	\$a1 00000000
> [13][31:0]	00000000	\$a2 00000000
> [14][31:0]	00000000	\$a3 00000000
> [15][31:0]	00000000	\$t0 0000030c
> [16][31:0]	00000000	\$t1 00000028
> [17][31:0]	00000000	\$t2 00000028
> [18][31:0]	00000000	\$t3 00000000
> [19][31:0]	00000000	\$t4 00000000
> [20][31:0]	00000000	\$t5 00000000
> [21][31:0]	00000000	\$t6 00000000
> [22][31:0]	00000000	\$t7 00000000
> [23][31:0]	00000000	\$s0 00000000
> [24][31:0]	00000000	\$s1 00000000
> [25][31:0]	00000000	\$s2 00000000
> [26][31:0]	00000000	\$s3 00000000
> [27][31:0]	00000000	\$s4 00000000
> [28][31:0]	00000000	\$s5 00000000
> [29][31:0]	00000000	\$s6 00000000
> [30][31:0]	00000000	\$s7 00000000
> [31][31:0]	00000000	\$t8 00000000
> [32][31:0]	00000000	\$t9 00000000
> [33][31:0]	00000000	\$k0 00000000
> [34][31:0]	00000000	\$k1 00000000
> [35][31:0]	00001800	\$gp 00001800
> [36][31:0]	00003ffc	\$sp 00003ffc
> [37][31:0]	00000000	\$fp 00000000
> [38][31:0]	00000000	\$ra 00000000

Simulation success!!!

testcase4 - 성공

> [0][31:0]	00000000	havruta@BOOK-TCTP
> [1][31:0]	c0bf0000	u testcase4.hex
> [2][31:0]	00000000	Loading instruction
> [3][31:0]	00000000	c
> [4][31:0]	00000000	Starting CPU simu
> [5][31:0]	00000000	Simulation done s
> [6][31:0]	00000000	RF states after t
> [7][31:0]	00000000	\$zero 00000000
> [8][31:0]	c0bf0000	\$at c0bf0000
> [9][31:0]	00000000	\$v0 00000000
> [10][31:0]	ccea353c	\$v1 00000000
> [11][31:0]	5d5f78a6	\$a0 00000000
> [12][31:0]	b25816d9	\$a1 00000000
> [13][31:0]	996e0934	\$a2 00000000
> [14][31:0]	6445d3d1	\$a3 00000000
> [15][31:0]	d5cd61f4	\$t0 ccea353c
> [16][31:0]	00000000	\$t1 5d5f78a6
> [17][31:0]	00000000	\$t2 b25816d9
> [18][31:0]	00000000	\$t3 996e0934
> [19][31:0]	00000000	\$t4 6445d3d1
> [20][31:0]	c917ab77	\$t5 d5cd61f4
> [21][31:0]	00000000	\$t6 00000000
> [22][31:0]	00000000	\$t7 00000000
> [23][31:0]	00000000	\$s0 c917ab77
> [24][31:0]	00000000	\$s1 00000000
> [25][31:0]	00000000	\$s2 00000000
> [26][31:0]	00000000	\$s3 00000000
> [27][31:0]	00000000	\$s4 00000000
> [28][31:0]	00000000	\$s5 00000000
> [29][31:0]	00000000	\$s6 00000000
> [30][31:0]	00000000	\$s7 00000000
> [31][31:0]	00000000	\$t8 00000000
> [32][31:0]	00000000	\$t9 00000000
> [33][31:0]	00000000	\$k0 00000000
> [34][31:0]	00000000	\$k1 00000000
> [35][31:0]	00001800	\$gp 00001800
> [36][31:0]	00003ffc	\$sp 00003ffc
> [37][31:0]	00000000	\$fp 00000000
> [38][31:0]	00000000	\$ra 00000000

Simulation success!!!

testcase5 - 실패 → 성공

> [0][31:0]	00000000	havruta@BOOK-TCTP
> [1][31:0]	224d0000	u testcase5.hex
> [2][31:0]	00000005	Loading instructions
> [3][31:0]	00000000	c
> [4][31:0]	00000000	Starting CPU simulation
> [5][31:0]	00000000	Simulation done successfully
> [6][31:0]	00000000	RF states after test
> [7][31:0]	00000000	\$zero 00000000
> [8][31:0]	74e250d2	\$at 224d0000
> [9][31:0]	fbd5a118	\$v0 00000005
> [10][31:0]	a16cc4d0	\$v1 00000000
> [11][31:0]	5efc37c0	\$a0 00000000
> [12][31:0]	638f9338	\$a1 00000000
> [13][31:0]	e6442b6e	\$a2 00000000
> [14][31:0]	00000000	\$a3 00000000
> [15][31:0]	00000000	\$t0 74e250d2
> [16][31:0]	e219cc86	\$t1 fbd5a118
> [17][31:0]	11bba2a	\$t2 a16cc4d0
> [18][31:0]	65a4c2ac	\$t3 5efc37c0
> [19][31:0]	13d0e99e	\$t4 638f9338
> [20][31:0]	61116734	\$t5 e6442b6e
> [21][31:0]	fdead08c	\$t6 00000000
> [22][31:0]	00000000	\$t7 00000000
> [23][31:0]	00000000	\$s0 e219cc86
> [24][31:0]	00000000	\$s1 11bba2a
> [25][31:0]	00000000	\$s2 65a4c2ac
> [26][31:0]	00000000	\$s3 13d0e99e
> [27][31:0]	00000000	\$s4 61116734
> [28][31:0]	00000000	\$s5 fdead08c
> [29][31:0]	00000000	\$s6 00000000
> [30][31:0]	00000000	\$s7 00000000
> [31][31:0]	00000034	\$t8 00000000

Simulation success!!!

testcase6 - 성공

> [0][31:0]	00000000	havruta@BOOK-TCTP
> [1][31:0]	aa210000	u testcase6.hex
> [2][31:0]	00000005	Loading instructions
> [3][31:0]	00000000	c
> [4][31:0]	00000000	Starting CPU simulation
> [5][31:0]	00000000	Simulation done successfully
> [6][31:0]	00000000	RF states after test
> [7][31:0]	00000000	\$zero 00000000
> [8][31:0]	d7c716b4	\$at aa210000
> [9][31:0]	d7c716b4	\$v0 00000005
> [10][31:0]	00000000	\$v1 00000000
> [11][31:0]	00000000	\$a0 00000000
> [12][31:0]	00000000	\$a1 00000000
> [13][31:0]	00000000	\$a2 00000000
> [14][31:0]	00000000	\$a3 00000000
> [15][31:0]	00000000	\$t0 00003457
> [16][31:0]	00000000	\$t1 d7c716b4
> [17][31:0]	00000000	\$t2 00000000
> [18][31:0]	00000000	\$t3 00000000
> [19][31:0]	00000000	\$t4 00003457
> [20][31:0]	00000000	\$t5 00000000
> [21][31:0]	00000000	\$t6 00000000
> [22][31:0]	00000000	\$t7 00000000
> [23][31:0]	00000000	\$s0 00000000
> [24][31:0]	00000000	\$s1 00000000
> [25][31:0]	00000000	\$s2 00000000
> [26][31:0]	00000000	\$s3 00000000
> [27][31:0]	00000000	\$s4 00000000
> [28][31:0]	00000000	\$s5 00000000
> [29][31:0]	00000000	\$s6 00000000
> [30][31:0]	00000000	\$s7 00000000
> [31][31:0]	00000000	\$t8 00000000

Simulation success!!!

testcase7

> [0][31:0]	00000000	havruta@B00K-TCTPSJLV
> [1][31:0]	e2810000	u testcase7.hex
> [2][31:0]	00000000	Loading instruction m
> [3][31:0]	00000000	c
> [4][31:0]	00000000	Starting CPU simulati
> [5][31:0]	00000000	Simulation done succe
> [6][31:0]	00000000	RF states after the p
> [7][31:0]	00000000	\$zero 00000000
> [8][31:0]	00000000	\$at e2810000
> [9][31:0]	00000000	\$v0 00000000
> [10][31:0]	00000000	\$v1 00000000
> [11][31:0]	00000000	\$a0 00000000
> [12][31:0]	00000000	\$a1 00000000
> [13][31:0]	00000000	\$a2 00000000
> [14][31:0]	00000000	\$a3 00000000
> [15][31:0]	00000000	\$t0 00000000
> [16][31:0]	00000000	\$t1 00000000
> [17][31:0]	00000000	\$t2 00000000
> [18][31:0]	00000000	\$t3 00000000
> [19][31:0]	00000000	\$t4 00000000
> [20][31:0]	00000000	\$t5 00000000
> [21][31:0]	00000000	\$t6 00000000
> [22][31:0]	00000000	\$t7 00000000
> [23][31:0]	00000000	\$s0 00000000
> [24][31:0]	00000000	\$s1 00000000
> [25][31:0]	00000000	\$s2 00000000
> [26][31:0]	00000000	\$s3 00000000
> [27][31:0]	00000000	\$s4 00000000
> [28][31:0]	00000000	\$s5 00000000
> [29][31:0]	00000000	\$s6 00000000
> [30][31:0]	00000000	\$s7 00000000
> [31][31:0]	00000000	\$t8 00000000
> [32][31:0]	00000000	\$t9 00000000
> [33][31:0]	00000000	\$k0 00000000
> [34][31:0]	00000000	\$k1 00000000
> [35][31:0]	00001800	\$gp 00001800
> [36][31:0]	00003ffc	\$sp 00003ffc
> [37][31:0]	00000000	\$fp 00000000
> [38][31:0]	00000000	\$ra 00000000

Simulation success!!!

4. Trouble Shooting (v)

testcase2

4.1. MemtoReg가 작동하지 않는 것으로 보임.

기존에 로그를 찍어봐도, 값들이 XxxxXXXX와 같은 형식으로 나오며, 뭔가 초기화되지 않은 듯한 값들이 출력되고 있었다.

원활한 디버깅을 위해서는 이를 해결해야겠다고 생각했는데, 사실은 그 자체로 문제였다.

```
Cycle=15000, currentState=0, nextState=1, MemtoReg=z
Cycle=25000, currentState=1, nextState=2, MemtoReg=z
Cycle=35000, currentState=2, nextState=4, MemtoReg=z
[WB] PC=00000004, wr_addr= 1, wr_data=XXXXXXXX, ALUOut=00050000, MDR=3428bdcf, MemtoReg=z
Cycle=45000, currentState=4, nextState=0, MemtoReg=z
Cycle=55000, currentState=0, nextState=1, MemtoReg=z
Cycle=65000, currentState=1, nextState=2, MemtoReg=z
Cycle=75000, currentState=2, nextState=4, MemtoReg=z
[WB] PC=00000008, wr_addr= 8, wr_data=XXXXXXXX, ALUOut=XXXXbdcf, MDR=3c01fff9, MemtoReg=z
Cycle=85000, currentState=4, nextState=0, MemtoReg=z
Cycle=95000, currentState=0, nextState=1, MemtoReg=z
Cycle=105000, currentState=1, nextState=2, MemtoReg=z
Cycle=115000, currentState=2, nextState=4, MemtoReg=z
[WB] PC=0000000c, wr_addr= 1, wr_data=XXXXXXXX, ALUOut=fff90000, MDR=3429c2d1, MemtoReg=z
Cycle=125000, currentState=4, nextState=0, MemtoReg=z
```

위 사진처럼 MemtoReg에 값이 정해지지 않았다는 **z**가 출력되며, data들이 점점 망가지고 있었다.

코드를 다시 살펴보니, CPU.v 에서 CTRL.v 모듈과 연결하는 부분에서 문제가 있었다.

MemtoReg 를 선언만 해두고 연결하지 않아서 발생한 문제였다.

MemtoReg를 연결하니, 문제가 해결되었고, 정상적으로 시뮬레이션이 진행되었다.

```
        .IRWrite(IRWrite),  
        .RegDst(RegDst),  
        .RegWrite(RegWrite),  
        .MemtoReg(MemtoReg),  
        .ALUSrcA(ALUSrcA),  
        .ALUSrcB(ALUSrcB),  
        .ALUOp(ALUOp),  
        .PCSource(PCSource),  
        .SignExtend(SignExtend),  
        .nextState(nextState)
```

```
[WB] PC=00000a50, wr_addr= 2, wr_data=00000005, ALUOut=00000005, MDR=00000000, MemtoReg=0  
Cycle=7495000, currentState=4, nextState=0, MemtoReg=0  
Cycle=7505000, currentState=0, nextState=1, MemtoReg=0  
Program Terminate  
  
Simulation success!!!
```

testcase1도 우선 success가 출력되어서 넘어가긴 했었지만, CPU 속 RF 모듈을 살펴보면 값이 XXxxxXXX처럼 되어있어 의아해하고 있었다.

위의 수정사항으로 이제 RF에 있는 값들이 정상적으로 보인다.

testcase5

4.2 RF 결과는 동일한데, 시뮬레이션이 종료되지 않음.

Mars로 동작 순서를 확인해보니, jal 한번 발생하고, 쪽 명령어를 실행한 후, 뒤에서 두번째 명령어인 jr를 실행해서 상단으로 올라간 후, 다시 쪽 명령어를 실행하고 종료가 되어야한다.

```

PC=000004ec, IR=03e00008, halt=0
PC=000004ec, IR=03e00008, halt=0
PC=00000000, IR=03e00008, halt=0
PC=00000004, IR=3c01a14c, halt=0
PC=00000004, IR=3c01a14c, halt=0
PC=00000004, IR=3c01a14c, halt=0
PC=00000004, IR=3c01a14c, halt=0

```

하지만, 디버깅 결과, 계속해서 PC가 0으로 초기화되며 다시 실행되는 모습이다.

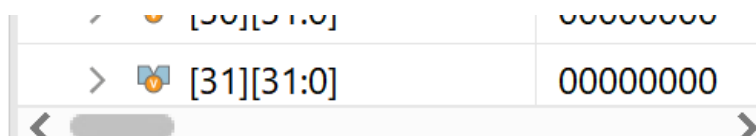
어떤 연산이 문제인가 싶어, 한번 클릭 시 100ns씩 작동되도록 하여 살펴보니, 문제가 확인되었다.

```

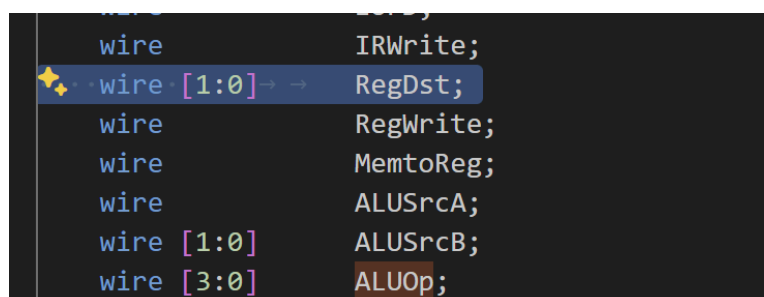
PC=000004ec, IR=03e00008, halt=0
run 100 ns
PC=00000000, IR=03e00008, halt=0

```

첫 JR 연산에서 ra에 저장된 34를 사용하는 것이 아니라, 0으로 초기화하고있었다.



확인 결과, JAL이 수행되었을 때, ra레지스터에 정상적으로 값이 적어지지 않고 있다.



RegDst 관련한 부분들이 1bit으로 설정되어 있어서, `wr_addr` 이 31번 레지스터를 가리키지 않았었다.

```

/ wr_addr=1f,

```

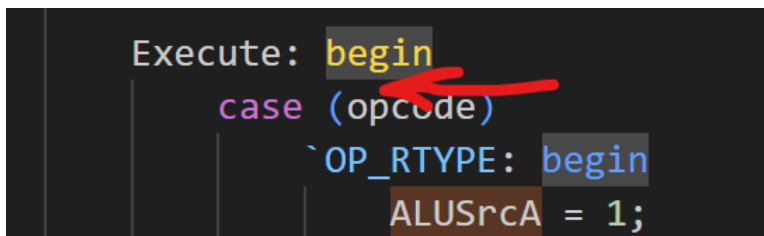
이 부분을 수정한 이후, 이 문제는 해결되었지만, ALUOut에 00000000값이 남아있는 문제는 여전히 존재했다.

```
[JAL1] PC=00000034, state=1, RegWrite=0, RegDst=0, MemtoReg=0, alu_result=000002ac, ALUOut=00000034 <-prev ex result
[JAL2] ALUSrcA=0, ALUSrcB=3
[JAL3] PCWrite=0, PCSource=0 // wr_addr=00, wr_data=00000034
[JAL1] PC=00000034, state=2, RegWrite=0, RegDst=0, MemtoReg=0, alu_result=00000000, ALUOut=000002ac <-prev ex result
[JAL2] ALUSrcA=1, ALUSrcB=0
[JAL3] PCWrite=1, PCSource=2 // wr_addr=00, wr_data=000002ac
[JAL1] PC=00000278, state=4, RegWrite=1, RegDst=2, MemtoReg=0, alu_result=00000278, ALUOut=00000000 <-prev ex result
[JAL2] ALUSrcA=0, ALUSrcB=0
[JAL3] PCWrite=0, PCSource=0 // wr_addr=if, wr_data=00000000
[JAL1] PC=00000278, state=0, RegWrite=0, RegDst=0, MemtoReg=0, alu_result=0000027c, ALUOut=00000278 <-prev ex result
[JAL2] ALUSrcA=0, ALUSrcB=1
[JAL3] PCWrite=1, PCSource=0 // wr_addr=00, wr_data=00000278
```

WB단계에서 ALUOut으로 적을 것을 생각하면

Ex단계에서 alu_result에 ra에 저장할 값이 존재해야 하는데..

아 OP_JAL의 Execution단계에서만은 ALUSrcA에 1을 할당하지 않았어야 했는데, 전부 다 1인 줄 알고, switch구문의 최상단에 일괄적으로 해당 값을 처리하고 있었다. 해당 부분을 수정하고 나서, 문제가 해결되었다!



```
Execute: begin
  case (opcode)
    OP_RTYPE: begin
      ALUSrcA = 1;
```

해당 위치에 ALUSrcA = 1이 존재했다.

Simulation success!!!